



COMPSCI 130 S1 2022

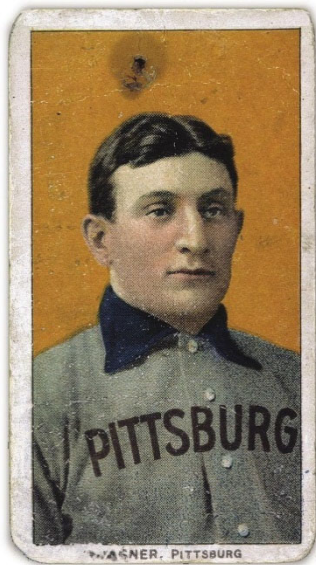
Introduction to Software Fundamentals

Hashtables - Introduction



Why hashtables?

- ▶ Consider searching for a particular card in a large stack of unordered cards



1909 T206 Honus Wagner
Sold US \$2.8 million (2007)



this is $O(n)$

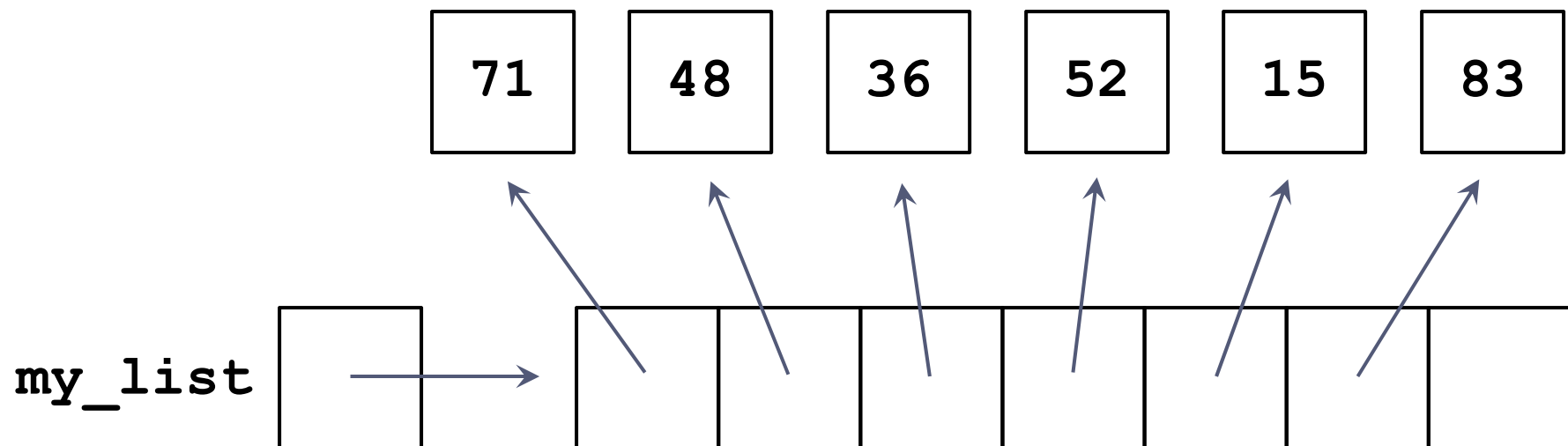


Why hashtables?

- Recall, we implemented the List ADT for **unordered** data using a Python list

Python list implementation of unordered List ADT

Operation	Cost
add(item)	$O(n)$
search(item)	$O(n)$
remove(item)	$O(n)$





Why hashtables?

- ▶ Consider searching for a name in a phone book:

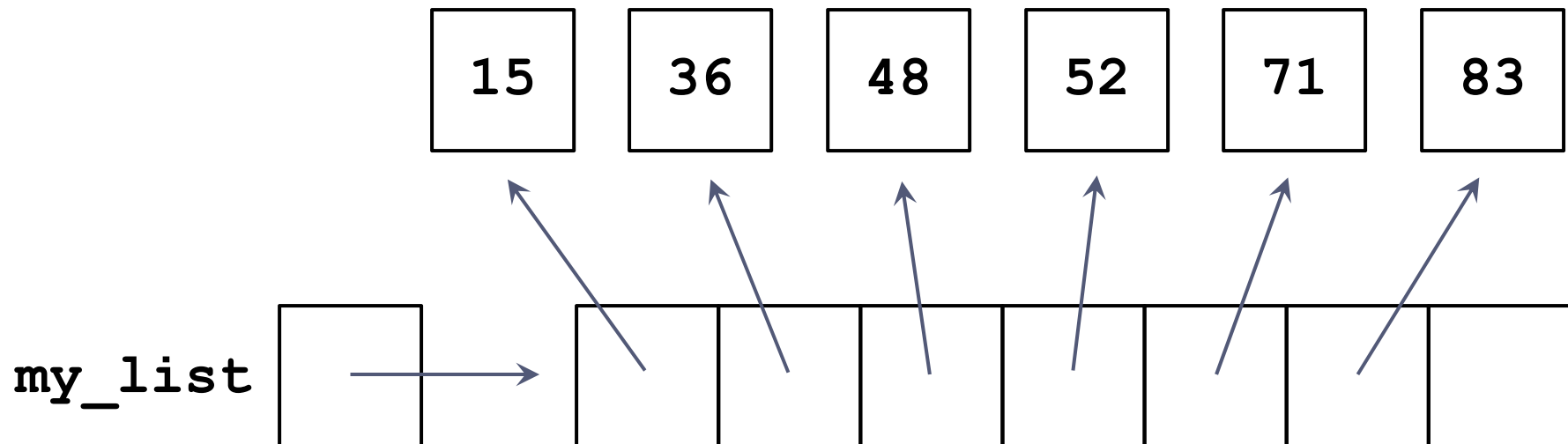


this is $O(\log n)$ (binary search)



Why hashtables?

- ▶ If we have **ordered data**, we can use a **binary search** to reduce searching time from $O(n)$ to $O(\log n)$
- ▶ Let's consider a Python list implementation of the List ADT where we maintain items **in order**





Why hashtables?

- ▶ If we have **ordered data**, we can use a binary search to reduce searching time from $O(n)$ to $O(\log n)$
- ▶ Let's consider a Python list implementation of the List ADT where we maintain items **in order**
 - ▶ **add(item)** – can use a binary search to find where to insert the item so that all items remain in order
 - ▶ **search(item)** – can use a binary search to locate an item
 - ▶ **remove(item)** – can use a binary search to locate the item to remove
- ▶ What are the costs (expressed in big-O) for these operations?

Python list implementation of ordered List ADT

Operation	Cost
add(item)	$O(n)$
search(item)	$O(\log n)$
remove(item)	$O(n)$

there is an $O(n)$ cost associated with shifting items in the Python list



Hashtables

- ▶ Can we do better?
 - ▶ Can we do better than $O(\log n)$ *search*?
 - ▶ Can we do better than $O(n)$ *add* and *remove*?



Hashtables

Experiment: searching list vs. dictionary

```
my_map = dict()
my_list = list()

for i in range(0, 10):
    my_list.append(str(i))

for i in range(0, 10):
    my_map[str(i)] = i

print(my_list)
print(my_map)
```

The 'in' operator searches for a *value* stored in a list, or a *key* stored in a dictionary

```
print('5' in my_list)
print('5' in my_map)

print('xyz' in my_list)
print('xyz' in my_map)
```



```
True
True
False
False
```



```
['0', '1', '2', '3', '4', '5', '6', '7', '8', '9']
```

```
{'2': 2, '5': 5, '7': 7, '4': 4, '1': 1, '6': 6, '0': 0, '9': 9, '3': 3, '8': 8}
```




Hashtables

► Two timed experiments:

'xyz' in my_list

```
def time_search(n):  
  
    my_list = list()  
    print(n, end = ' ')  
  
    for i in range(0, n):  
        my_list.append(str(i))  
  
    time_start = time.time()  
  
    if ('xyz' in my_list):  
        print('Found')  
  
    time_end = time.time()  
    time_taken = time_end - time_start  
  
    print(time_taken)  
  
for i in range(1000000, 65000000, 1000000):  
    time_search(i)
```

'xyz' in my_map

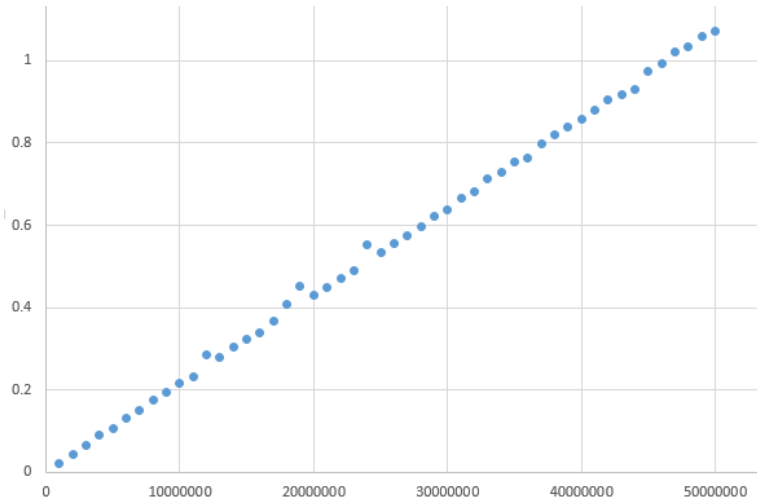
```
def time_search(n):  
  
    my_map = dict()  
    print(n, end = ' ')  
  
    for i in range(0, n):  
        my_map[str(i)] = i  
  
    time_start = time.time()  
  
    if ('xyz' in my_map):  
        print('Found')  
  
    time_end = time.time()  
    time_taken = time_end - time_start  
  
    print(time_taken)  
  
for i in range(1000000, 65000000, 1000000):  
    time_search(i)
```



Hashtables

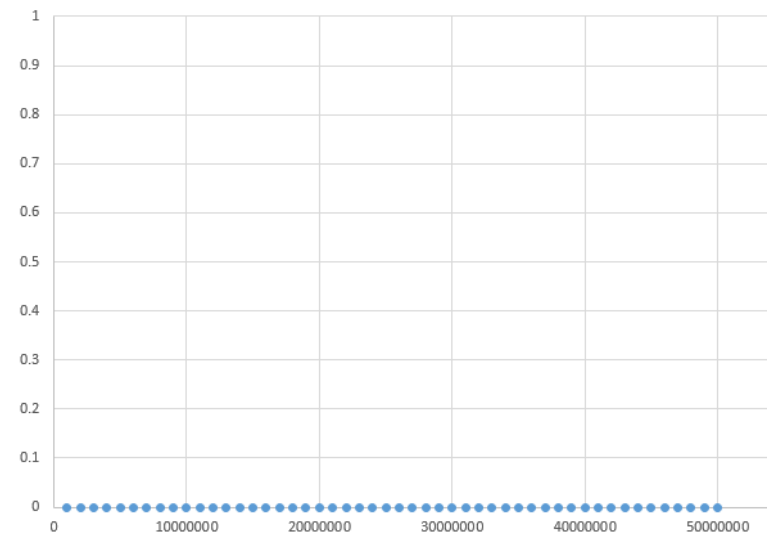
► Two timed experiments:

'xyz' in my_list



This is $O(n)$

'xyz' in my_map



This is $O(1)$



Hashtables

- ▶ Can we do better?
 - ▶ Can we do better than $O(\log n)$ *search*?
 - ▶ Can we do better than $O(n)$ *add* and *remove*?

Hashtable

Operation	Cost
add(item)	$O(1)$
search(item)	$O(1)$
remove(item)	$O(1)$